

Desarrollo del Gemelo Digital del Beamline: Implementación de la Fase 2 (Emulador Forward)

Proyecto MIT - UNAL, Medellín
Hackathon Digital Twin

8 de julio de 2026

Índice

1. Introducción	2
2. Fundamentación Teórica del Emulador Forward	2
2.1. ¿Qué es el Emulador Forward?	2
2.2. ¿Por qué mapear al Espacio Latente en lugar de mapear directo?	2
2.3. ¿Para qué sirve esta arquitectura? (Rúbrica D)	3
3. Arquitectura Neuronal del Emulador	3
3.1. Forward Regressor (DNN)	3
3.2. Full Emulator (Compuesto)	3
4. Proceso de Entrenamiento y Datos	3
4.1. Codificación Latente de los Datos de Entrenamiento	4
4.2. Función de Pérdida y Optimización	4
5. Base de Código de la Fase 2	4
6. Resultados y Verificación	4
6.1. Curva de Convergencia	4
6.2. Cálculo Analítico de Sensibilidad (Gradientes)	5

1. Introducción

Este documento describe el desarrollo y la implementación en código de la **Fase 2: El Emulador Forward (DNN)** en el marco del Gemelo Digital del beamline de iones. La arquitectura sigue la filosofía del paper *CBOL-Tuner* de Rautela et al., donde el modelado predictivo del sistema no se realiza mapeando directamente las acciones físicas a los estados complejos (de muy alta dimensión y ruidosos), sino reduciendo el mapeo a través del espacio latente entrenado en la Fase 1.

A continuación, se justifica la fundamentación matemática, la arquitectura de la red implementada, el proceso de entrenamiento y la verificación práctica de la diferenciabilidad del emulador.

2. Fundamentación Teórica del Emulador Forward

En un beamline real o simulado mediante SIMION, el comportamiento del sistema está determinado por un vector de voltajes de entrada (acciones $y \in \mathbb{R}^8$) y produce una distribución de partículas en el detector (estado $X \in \mathbb{R}^{400}$).

2.1. ¿Qué es el Emulador Forward?

El Emulador Forward es un regresor $f_\psi(y)$ estructurado como una red neuronal artificial profunda (DNN) que recibe los voltajes normalizados y predice las coordenadas en el espacio latente del VAE:

$$f_\psi : y \mapsto \hat{z} \in \mathbb{R}^2 \quad (1)$$

El flujo completo de predicción para reconstruir el perfil espacial del haz en el detector es:

$$y \xrightarrow{f_\psi} \hat{z} \xrightarrow{\text{Decoder}} \hat{X} \quad (2)$$

2.2. ¿Por qué mapear al Espacio Latente en lugar de mapear directo?

Mapear directamente los voltajes a la distribución del detector ($y \rightarrow X$) presenta múltiples inconvenientes:

- **Alta dimensionalidad y escasez de datos:** Mapear 8 entradas a 400 salidas independientes requiere una gran cantidad de parámetros en la red, lo que induce sobreajuste con pocos datos.
- **Predicciones físicamente imposibles:** Una red que mapee directamente a X no conoce las correlaciones espaciales reales y puede predecir ruido de fondo o distribuciones no físicas (hallucinations).
- **Restricción física (Manifold Latente):** Al forzar a que el regresor apunte a la zona latente z del VAE, las predicciones del decodificador están restringidas al subespacio (manifold) de distribuciones que el VAE aprendió como “físicamente posibles” durante la Fase 1.

2.3. ¿Para qué sirve esta arquitectura? (Rúbrica D)

El fin de este diseño es la **Diferenciabilidad Analítica de Extremo a Extremo**. Como el regresor f_ψ y el Decodificador del VAE están implementados en PyTorch, el flujo compuesto completo:

$$\text{Score} = g(\text{Decoder}(f_\psi(y))) \quad (3)$$

se compone enteramente de operaciones diferenciables. Esto nos permite calcular analíticamente y de manera instantánea el gradiente del perfil de haz predicho con respecto a las perillas físicas de entrada:

$$\nabla_y \text{Score} = \frac{\partial \text{Score}}{\partial y} \quad (4)$$

usando retropropagación (`.backward()`). Gracias a este gradiente, es posible optimizar el enfoque y alineación del haz de forma determinista usando optimizadores de primer orden en milisegundos, en lugar de realizar miles de simulaciones a ciegas en SIMION.

3. Arquitectura Neuronal del Emulador

La red se implementa en PyTorch en el archivo `model.py` y se compone de dos elementos fundamentales:

3.1. Forward Regressor (DNN)

El regresor es un perceptrón multicapa (MLP) con la siguiente estructura de capas densas y activaciones lineales/ReLU:

$$8 \xrightarrow{\text{Linear+ReLU}} 64 \xrightarrow{\text{Linear+ReLU}} 128 \xrightarrow{\text{Linear+ReLU}} 64 \xrightarrow{\text{Linear}} 2 \quad (5)$$

No se incluye Batch Normalization aquí para garantizar que la salida de la función sea puramente determinista y suave respecto al vector de voltajes de entrada y .

3.2. Full Emulator (Compuesto)

La clase `FullEmulator` amarra ambas fases en una única tubería de ejecución de PyTorch. Al instanciarse, recibe el regresor entrenado y el Decodificador del VAE. De forma automática, congela todos los parámetros del Decodificador:

```
1 for param in self.decoder.parameters():
2     param.requires_grad = False
```

Esto garantiza que durante el ajuste del emulador forward, no se modifique el espacio latente del VAE, optimizando únicamente el mapeo del regresor.

4. Proceso de Entrenamiento y Datos

El entrenamiento está gestionado por el archivo `train_dnn.py`.

4.1. Codificación Latente de los Datos de Entrenamiento

Dado que el dataset en formato comprimido de NumPy `beamline_dataset.npz` contiene los voltajes y_i y los histogramas reales X_i , primero pasamos todos los histogramas X_i por el Encoder del VAE cargado para obtener la distribución latente determinista $z_i = \mu_{z,i}$. De esta forma, construimos pares de entrenamiento (y_i, z_i) .

4.2. Función de Pérdida y Optimización

La pérdida que se minimiza para entrenar el regresor es el Error Cuadrático Medio (MSE) en el espacio latente:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B} \sum_{b=1}^B \|\hat{z}_b - z_b\|^2 \quad (6)$$

Se divide el dataset en Entrenamiento y Validación (80/20). Se utiliza el optimizador Adam con una tasa de aprendizaje de 10^{-3} y un tamaño de lote de 8.

5. Base de Código de la Fase 2

Los códigos correspondientes se ubican en la carpeta `src/phase_2_DNN/`:

- `model.py`: Contiene las clases `ForwardRegressor` y `FullEmulator`.
- `train_dnn.py`: Carga el dataset, el VAE de la Fase 1, realiza la codificación latente, entrena el regresor y guarda los pesos en `forward_regressor.pt`.
- `verify_emulator.py`: Script de diagnóstico que verifica de manera práctica la diferenciabilidad del modelo calculando gradientes de la transmisión del haz respecto a los voltajes.

6. Resultados y Verificación

El regresor forward se entrenó sobre las 46 simulaciones recolectadas de SIMION.

6.1. Curva de Convergencia

El entrenamiento de 150 épocas en CUDA arrojó los siguientes resultados:

- **Época 1**: MSE de entrenamiento de 0,037108, validación de 0,008147.
- **Época 15**: MSE de entrenamiento de 0,000163, validación de 0,000323.
- **Época 150**: Finalización exitosa con un MSE de validación óptimo de **0.000155**, guardándose en `forward_regressor.pt`.

La bajísima pérdida MSE latente indica que el regresor modela con alta fidelidad las coordenadas latentes correspondientes a cada combinación física de voltajes.

6.2. Cálculo Analítico de Sensibilidad (Gradientes)

Se ejecutó el script de verificación cargando el emulador compuesto. Introduciendo voltajes de prueba normalizados se predijo el perfil de haz $\hat{X}$ y se definió la transmisión como la suma de bins del histograma: $\text{Transmission} = \sum \hat{X}_i$. Aplicando retropropagación, obtuvimos los gradientes analíticos reales en consola:

Cuadro 1: Gradientes de sensibilidad obtenidos mediante retropropagación.

Voltaje del Electrodo	Sensibilidad Analítica ($\partial \text{Transmisión} / \partial V$)
V_3 (Lente Einzel 1 centro)	-0,019970
V_6 (Lente Einzel 2 centro)	-0,023670
V_9 (Curvador cuadrupolar 1)	+0,070732
V_{10} (Curvador cuadrupolar 2)	-0,024156
V_{11} (Curvador cuadrupolar 3)	+0,022326
V_{12} (Curvador cuadrupolar 4)	-0,067548
V_{15} (Placa dirección 1)	+0,055127
V_{18} (Placa dirección 2)	+0,001081

Estos gradientes muestran, por ejemplo, que para incrementar la transmisión, se debería aumentar ligeramente el voltaje en el electrodo 9 (V_9) y disminuir el de la lente Einzel 1 (V_3). Esto valida la completa diferenciabilidad y la utilidad de control del Gemelo Digital.